

Introduction to Infrastructure as Code & Terraform

Why we describe infrastructure as code
— and the core Terraform workflow you
will use for the next 20 hours.

Infrastructure as Code

Terraform & HCL

Providers

Core workflow

Lab 01

What we'll cover today



1 - Infrastructure as Code

What IaC is, the benefits, declarative vs imperative, and where Terraform sits in the tooling landscape.



2 - Meet Terraform

HCL, providers, and the init → validate → plan → apply → destroy workflow that repeats all course long.



3 - Demo

Run your first Terraform commands live against the GitHub provider — no cloud account required.



4 - Lab 01

Create the Git repository that will host your project for the whole course — with Terraform itself.

01

Infrastructure as Code

Treat servers, networks and cloud services the same way you treat application source code.

What is Infrastructure as Code?

Defining your infrastructure through **source code** that is treated like any other software system.

- Compute, networking, security, managed services — even **GitHub repos**
- Kept in **source control** for auditability, versioning and CI/CD
- Natural fit for cloud, but works on-prem too

HCL

```
resource "azurerm_resource_group" "app" {  
  name      = "app-prod-rg"  
  location  = "westeurope"  
  
  tags = {  
    environment = "prod"  
    managed_by  = "terraform"  
  }  
}
```

💡 The code **is** the documentation — it always reflects what is really deployed.

The benefits



Fast & repeatable

Provision, validate and reconfigure infra quickly and consistently every time.



No config drift

One source of truth keeps every environment consistent and predictable.



Cost control

Spin dynamic environments up and down on demand instead of leaving them running.



Versioned

Lives beside your app code and pipelines, with full history and review.



Live documentation

Declarative config doubles as always-current infrastructure docs.



Testable & shift-left

Validate and security-scan your infra definition before anything is created.

Declarative, not imperative

Declarative — "the what"

You describe the **desired end state** in a human-readable way. The tool figures out how to get there.

Idempotent: apply it once or a hundred times — the outcome is always the same.

Imperative — "the how"

You script every step yourself. You **can** reach the same state, but you own all the sequencing and error handling.

Classic shell / CLI scripting.

- ✓ Terraform is **declarative**: you state the goal, Terraform builds and maintains the plan to reach it.

IaC vs. Configuration Management

INFRASTRUCTURE AS CODE

Defines the infrastructure resources

Compute, networking, managed services

Terraform, Pulumi, ARM/Bicep

CONFIGURATION MANAGEMENT

Configures software **inside** the infrastructure

Packages, services, files on a machine

Ansible, Chef, Puppet

💡 They are complementary — often used **together** for a fully automated environment. CM tools can create infra too, but with limited capabilities.

Tooling: proprietary vs. provider-agnostic

Proprietary

Tied to one provider — Azure **ARM / Bicep**, AWS **CloudFormation**, GCP **Deployment Manager**.

Provider-agnostic

Not coupled to one cloud — **Terraform**, Pulumi, Ansible.

➤ **Agnostic — pros**

- Better for hybrid environments
- Bigger communities & ecosystem
- One language across providers

▲ **Trade-offs**

- Feature parity can lag the provider
- Switching provider isn't just config

02

Meet Terraform

Multi-platform, multi-provider IaC from HashiCorp
— with the biggest ecosystem in the space.

What is Terraform?



Multi-provider

One tool to build across Azure, AWS, GCP, GitHub and hundreds more via providers.



Huge ecosystem

The largest IaC community, with providers and modules for almost anything.



A single binary

Written in Go, cross-platform. Download, unzip, run — that's the install.



Clean language

Uses HCL — purpose-built, readable, and similar in feel to JSON / YAML.

HCL & Providers

HCL — the language

- A small domain-specific language, JSON-compatible
- Describes resources and their relationships

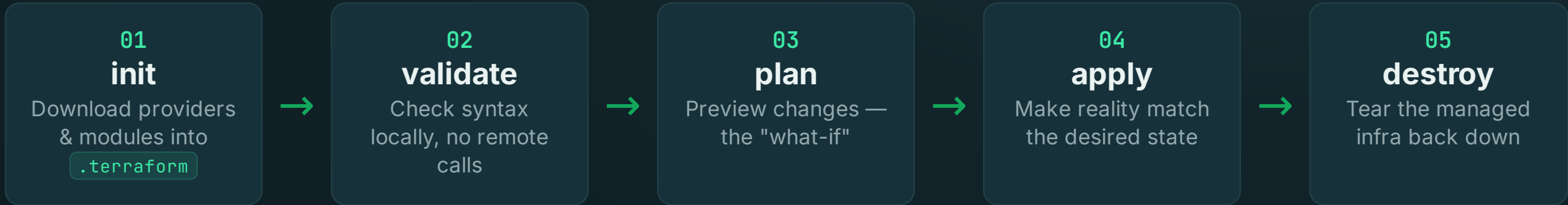
Providers — how Terraform talks to the world

- Plugins for each platform (Azure, GitHub, ...) — anyone can build one for a CLI/API
- Downloaded by `terraform init`

HCL

```
terraform {  
  required_providers {  
    github = {  
      source  = "integrations/github"  
      version = "~> 6.0"  
    }  
  }  
}  
  
provider "github" {}  
  
resource "github_repository" "course" {  
  name          = "terraform-training"  
  visibility    = "private"  
}
```

The core Terraform workflow



🔄 Re-run `init` whenever you add or change a provider or module. Everything else is just this loop, over and over.

What each step really does

terraform init

Fetches required providers/modules into `.terraform` and writes the lock file. Always run it first.

terraform validate

Confirms your `.tf` files are syntactically valid — no connection to the provider needed.

terraform plan

Shows the exact set of create / change / destroy actions before you commit to anything.

terraform apply

Runs a plan, then — on your approval — applies it to reach the desired state.

⚠ `terraform destroy` removes all Terraform-managed infrastructure. Powerful — and irreversible.

 LIVE DEMO

Run your first Terraform commands

Create a real GitHub repository from HCL — **init** → **plan** → **apply** — then prove idempotence by applying again.

Provider · GitHub **Auth** · GitHub CLI **Cost** · Free

🔗 LAB 01 • 30-40 MIN

Create your course repository with Terraform

Author your first configuration and use Terraform to create the Git repo that will host your project for the rest of the course.

You'll practise: authoring HCL · `init` · `validate` · `plan` · `apply` · reading state · `destroy`

Key takeaways

✓ IaC = infra as versioned code

Declarative, idempotent, source-controlled, and self-documenting.

✓ Terraform is provider-agnostic

One language (HCL) + providers to build across many platforms.

✓ The workflow is the loop

init → validate → plan → apply → destroy — you'll live in it.

✓ Next up

Session 02 — providers, resources, dependencies and Terraform state.



Questions?

Up next — [Session 02: Main Concepts](#).
Take the break, then we build.